

Applied Software Project Report

**ArogyaDesk: A Concurrent Appointment and OPD Queue Platform for
Multi-Clinic Outpatient Care**

By

Sobhanbabu Katikapalle

A Master's Project Report submitted to Scaler Neovarsity - Woolf in partial
fulfillment of the requirements for the degree of Master of Science in Computer
Science

September 2026

Scaler Mentee Email ID : sobhanbabu.katikapalle@gmail.com

Thesis Supervisor : Naman Bhalla

Date of Submission : 28/09/2026

Certification

I confirm that I have overseen / reviewed this applied project and, in my judgment, it adheres to the appropriate standards of academic presentation. I believe it satisfactorily meets the criteria, in terms of both quality and breadth, to serve as an applied project report for the attainment of Master of Science in Computer Science degree. This applied project report has been submitted to Woolf and is deemed sufficient to fulfill the prerequisites for the Master of Science in Computer Science degree.

Naman Bhalla

.....

Project Guide / Supervisor

DECLARATION

I confirm that this project report, submitted to fulfill the requirements for the Master of Science in Computer Science degree, completed by me from 2 March 2024 to 21 May 2024, is the result of my own individual endeavor. The Project has been made on my own under the guidance of my supervisor with proper acknowledgement and without plagiarism. Any contributions from external sources or individuals, including the use of AI tools, are appropriately acknowledged through citation. By making this declaration, I acknowledge that any violation of this statement constitutes academic misconduct. I understand that such misconduct may lead to expulsion from the program and/or disqualification from receiving the degree.

Sobhanbabu Katikapalle

Sobhanbabu Katikapalle

Date: 28 September 2026

ACKNOWLEDGMENT

I thank my family for the patience and practical help that made it possible to keep studying while working. I thank the instructors and mentors at Scaler, and my project supervisor Naman Bhalla, for the backend classes that shaped the way this system is divided into services, tables, and failure cases. I also thank the friends who asked awkward questions about double booking and about what a clinic actually does at the reception desk. Those questions changed the design more than any framework choice did.

The behaviour described in this report was checked by running the automated tests in the repository. The timing numbers in the feature chapter come from that run on the development machine. Official documentation for FastAPI, SQLAlchemy, MySQL, Redis, Kafka, JSON Web Tokens, and Amazon Web Services was consulted while writing the technology and deployment chapters, and those works are listed in the references. An AI-assisted editor in Cursor was used to help structure the prose and to review the implementation. That assistance is cited in the references and does not replace the design decisions, the tests, or the responsibility for this submission.

Table of Contents

Section	Page No.
Applied Software Project	9
Abstract	9
Project Description	9
The problem	9
Objectives	10
Scope and what is deliberately left out	11
Who the system is for	11
How the project can be judged	11
How the work was organised	12
The system in its surroundings	12
What a reviewer will find in the repository	13
Requirement Gathering	14
How the requirements were gathered	14
Functional requirements	14
Non-functional requirements	16
Actors	17
Use cases	18
Feature set	19
Assumptions and constraints	20
Class Diagrams	21
How the code is layered	21
AppointmentService in more detail	23
Appointment states	23
Sequence of a successful booking	25
The cache and the outbox as classes	26
Database Schema Design	26
Rules that shaped the tables	26
Tables	27
Keys, foreign keys, and cardinality	29
Indexes and the day guard	31
A worked morning	32

Section	Page No.
The conditional update in SQL	33
Feature Development Process	34
Why booking is the feature this chapter follows	34
The request path	34
What the service does, in order	35
Token, queue, and the people who have not arrived	36
Cache behaviour on this feature	37
What was measured	38
Tests that hold the feature in place	39
What was optimised, and what was refused	40
Deployment Flow	41
What this chapter is	41
VPC, subnets, and security groups	42
EC2, Elastic Beanstalk, and the image	43
RDS, ElastiCache, and MSK	43
The order of a release	44
An illustration of cost, not a quote	44
Technologies Used	45
Why these tools, in this project	45
Python and FastAPI	46
SQLAlchemy and MySQL	47
Redis	47
Kafka and the outbox	48
Passwords and JSON Web Tokens	48
Docker and the cloud services	49
Conclusion	49
What this project taught	49
Where the same design is useful	49
Limits and what should be done next	50
References	51

List of Tables

(To be written sequentially as they appear in the text)

Table No.	Title	Page No.
2.01	Functional requirements by area	16
2.02	Non-functional requirements	17
2.03	Actors and the work they are allowed to do	17
2.04	Feature set of the application	19
3.01	Class responsibilities	22
3.02	Allowed appointment transitions	24
4.01	Tables in the schema	28
4.02	Cardinality of the main relationships	30
4.03	Indexes that protect correctness or the hot read	31
4.04	Sample rows after Asha books the 10:00 slot	33
5.01	Booking request, success, and the conflict that matters	35
5.02	Measurements from the test run on 28 September 2026	38
5.03	Automated tests and the rule each one protects	40
6.01	Security groups in the deployment design	42
6.02	Illustrative monthly cost for a small pilot, not a quotation	44
7.01	Technologies and the job each one has in ArogyaDesk	45

List of Figures

(List of Images, Graphs, Charts sequentially as they appear in the text)

Figure No.	Title	Page No.
1.01	Project development process	12
1.02	System context of ArogyaDesk	13
2.01	Use cases of ArogyaDesk	19
3.01	Principal classes and the methods a reviewer will look for	22
3.02	Appointment states and the transitions the services allow	24
3.03	Sequence of a successful booking	25
4.01	Entities and the relationships between them	27
5.01	Path of a booking request through the API	35
5.02	The booking transaction and the outbox row	37
5.03	Average time to serve a slot list, logarithmic scale	39
6.01	Deployment design on AWS in a single region	41

Applied Software Project

Abstract

ArogyaDesk is a backend for the outpatient department of a clinic that still mixes a paper token tray with phone bookings and a messaging-app list. The software gives that clinic one record of which doctor is working, which quarter-hour slots are still open, who has arrived, and whether the consultation has been paid. A patient, a receptionist, a doctor, and a clinic administrator each see only the actions their role is allowed to perform. The same service can hold more than one clinic, because departments, doctors, and slots are stored under a clinic rather than in a single hard-coded timetable.

The difficult part is not the form that collects a name. The difficult part is the slot. Two people can request the last morning appointment in the same second. ArogyaDesk treats that request as a conditional update: the slot row changes from open to booked only if it is still open, and the appointment, the token, the audit row, and an outbox event are written in the same database transaction. If the update changes no row, the caller receives a conflict and no second appointment is stored. A separate guard stops one patient from holding two active visits with the same doctor on the same day.

Open slot lists are cached so the reception desk is not reading the slot table for every refresh. The cache entry is removed only after the booking transaction commits. Domain events are not sent to Kafka from inside the request. They are stored in an outbox table and published by a relay, which means a crash after the commit can still be recovered by publishing the row that was left behind. The automated tests include a visit from booking to a paid invoice, and a trial of sixteen overlapping bookings against one slot. On that trial exactly one booking succeeded. The project is a working API with a deployment design for Amazon Web Services. A live cloud account was not required to demonstrate the booking rule, and this report does not pretend otherwise.

Project Description

The problem

A small outpatient clinic in an Indian city often runs on a routine that the staff can recite and a stranger cannot see. The doctor arrives, the receptionist tears tokens from a booklet or types numbers into a sheet, and patients who called in the morning are written in the margin. Someone who is already in the waiting room and

someone who is still in traffic are not the same kind of waiting, but a paper list treats them as one column. When the popular physician's list fills, the next caller is told to try again, and two receptionists can both say yes to the same remaining time. The patient who arrives is then asked to wait without a number, or is turned away after travelling. The clinic did not lack effort. It lacked a single decision about whether a slot was still free.

The cost of that confusion is not only a rude morning. A missed slot is an empty chair in a room that was staffed and paid for. A double booking is a longer queue and a doctor who starts late. The prescription and the bill, when they exist, sit in another notebook, so the person who collects the fee has to ask the doctor what the visit was worth. None of this requires a hospital information system of the kind a large chain buys. It requires a small set of rules that a computer will apply the same way on a busy Monday and on a quiet Thursday.

ArogyaDesk is that set of rules, exposed as an HTTP API. It does not try to be the clinic's entire world. It does not store laboratory machines, insurance tariffs, or a pharmacy inventory. It stores the outpatient visit from the moment a slot is offered until the consultation fee is settled. The name is a working name for the project: a desk where the OPD is coordinated. The sample clinic used throughout this report is CityCare OPD in Hyderabad, with one general physician, Dr. Meera Iyer, a consultation fee of 500 rupees, and fifteen-minute slots. The sample is data created by the seed routine. It is not a claim about a real clinic of that name.

Objectives

The first objective is to keep a slot from being sold twice. Every other feature is easier to forgive than a double booking. The second objective is to separate people who have an appointment from people who have arrived. The doctor should call a patient who is in the building. The third objective is to connect the visit to a note, a short prescription, and a bill, so the fee is not negotiated from memory. The fourth objective is to leave a record of who did each of those things. The fifth is to structure the work so that a second clinic, a second department, or a second doctor does not require a new program, only new rows.

A further objective is educational and practical at the same time. The backend classes in this programme asked for a service boundary, a relational schema, a careful write path, a cache with an invalidation story, and a way to publish events without lying about the database commit. ArogyaDesk is the place those ideas are used together on one workflow, instead of being demonstrated on unrelated

exercises. The repository is organised so a reviewer can run the tests without MySQL, Redis, or Kafka, and can still read the production settings that turn those components on.

Scope and what is deliberately left out

Inside the scope are accounts and roles, clinics and departments, doctor profiles, weekly schedules, generation of slots, booking, cancellation, no-show, check-in, the token queue, the consultation note, prescription lines, a consultation invoice, and partial payments. Also inside the scope are a JSON Web Token for sign-in, a cache for open slots, an outbox table, and a relay that can publish to Kafka. The MySQL schema is written out in db/schema.sql. Docker Compose describes a local MySQL, Redis, and Kafka. A Dockerfile describes how the API process is packed.

Outside the scope are a mobile application, online payment capture through a gateway, medicine stock, laboratory orders, insurance claims, appointment reminders by SMS, and any clinical decision support. The bill is the doctor's consultation fee only. Medicines are named on the prescription and are not priced. A patient cannot yet edit a profile beyond what was stored at registration. There is no multi-factor authentication. These are real limits. They are recorded again in the conclusion so they are not mistaken for unfinished accidents.

Who the system is for

Four roles use the API. A patient creates an account, looks at open slots, books for himself or herself, cancels inside the allowed window, and can read his or her own appointments and invoice. A receptionist books on behalf of a patient who walked in or phoned, checks a patient in, marks a no-show, generates slots from the weekly pattern, and records cash, UPI, or card payments. A doctor reads only his or her own queue, calls the next patient who has already checked in, and writes the note and prescription for that consultation. An administrator creates the clinic, the departments, the staff accounts, and the weekly schedule, and can read the audit log and the cache counters.

The administrator is not a doctor and is not expected to write clinical notes. The doctor is not expected to take money. The patient is not expected to see the whole queue, because that list includes other people's names. These boundaries are enforced in the services, not only hidden in a screen that this project does not include. A future front end can be wrong and the API should still refuse the action.

How the project can be judged

The project is successful, for the purpose of this report, when five statements are true. A slot that is open becomes booked for exactly one appointment when many requests arrive together. A patient cannot hold two active appointments with the same doctor on the same day. A doctor cannot call a patient who has not checked in. Completing a consultation creates one invoice for the configured fee, and payments cannot exceed the remaining balance. The automated test suite passes on a clean machine with only Python installed. The measurements later in this report are from that suite, not from a spreadsheet of hoped-for numbers.

How the work was organised

The work followed a short path that is ordinary in a backend project and is worth writing down because it kept the risky part first. The problem and the roles were written before the tables. The tables were written before the booking method. The booking method was tested under overlap before the prescription and the bill were added. The cache and the outbox were added on the path that had already been tested, so they had to preserve the same result. The deployment chapter was written from the process and the data stores that the code actually uses, not from a diagram drawn first and then hoped for.

How ArogyaDesk was built

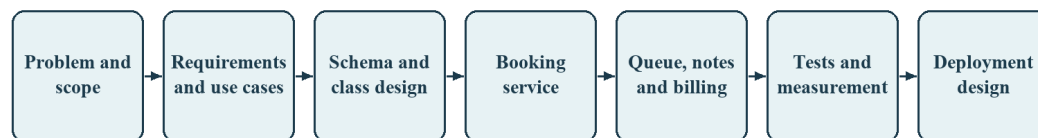


Figure 1.01: Project development process

Figure 1.01 is the order of that work. It is not a claim that every day was spent in a neat sequence. It is the order in which the design became hard to change. Once the conditional update and the day guard were in place, later features had to live with them. The queue, for example, does not invent a second way to number patients. It reads the token that booking already allocated.

The system in its surroundings

ArogyaDesk sits between four people and three stores. The people send HTTP requests. The API checks the token, calls a service, and lets the service talk to MySQL. Redis, when it is enabled, holds the open-slot list for a doctor and a date. Kafka, when it is enabled, receives the events that the outbox relay publishes. When

those two are not enabled, the API still runs: the slot cache is a process-local dictionary with the same methods, and the outbox rows wait unpublished until a relay is pointed at them. That split is what lets the tests run without extra services, and it is also what a single-clinic pilot can use on one machine before the clinic pays for a managed cache and a managed broker.

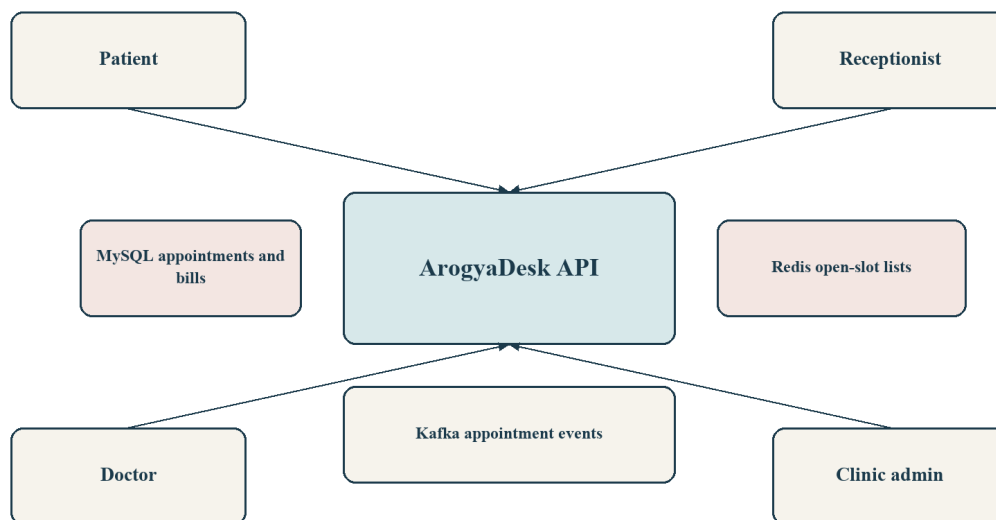


Figure 1.02: System context of ArogyaDesk

Figure 1.02 shows that context. The arrows are requests and stored records, not a promise that every box is running in production today. MySQL is the system of record. Redis is an optimisation that is allowed to be empty or wrong for at most the time-to-live, and is cleared on a successful booking or cancellation. Kafka is a notification path. A lost cache is repaired by reading MySQL. A lost Kafka message is repaired by publishing the outbox row again. A lost MySQL commit is not repaired by either of the other two, which is why the appointment and the outbox row share one transaction.

What a reviewer will find in the repository

The repository root contains the README, the Python package in `app`, the MySQL reference schema in `db/schema.sql`, the tests, a Dockerfile, and a Docker Compose file. The README lists the demo accounts, the main routes, and the command that runs the tests. Settings come from the environment. The sample JWT secret is refused when the environment is set to production. Passwords are stored as PBKDF2-HMAC-SHA256 hashes with a random salt. The seed routine creates CityCare OPD only when SEED is true and the clinic name is not already present,

so starting the process twice does not duplicate the doctor.

The code is divided the way the later class diagram describes. Routers accept and return JSON. Services apply the rules. SQLAlchemy models are the tables. There is no separate data-access class for every query, because the queries are few and they live next to the rule that needs them. That is a conscious choice for a project of this size. A larger team would pull the queries into repositories. The report's class diagram names the services a reader will actually open.

Requirement Gathering

How the requirements were gathered

The requirements did not come from a tender document. They came from watching the failure modes of a paper OPD and from the constraints of a backend project that has to be demonstrated. Each requirement below is written so that it can be tested. Where a requirement is a policy rather than a fact of nature, the policy is stated. A different clinic might want a different cancellation window or might want to bill a no-show. Those clinics would change a setting or a branch in the service. They would not need a different idea of a slot.

Functional requirements say what a role can do. Non-functional requirements say how the system should behave when it is slow, wrong, or attacked. Use cases then walk the same ground as stories, because a table of features hides the order of a morning. The feature catalogue is the short form of the same list, included because the template asks for the feature set in a table and because a reviewer should be able to find a feature without reading every paragraph.

Functional requirements

Identity comes first. A new patient can register with a name, an email address, and a password. The password must be between 8 and 72 characters and must contain a letter and a digit. Self-registration always creates a patient. It cannot create a doctor or an administrator, because that would let a stranger grant himself a clinical role. Staff accounts are created by an administrator. Sign-in returns a bearer token. A wrong password and an unknown email produce the same message, so the login form is not a directory of who has an account. Repeated failures from the same client address are rejected after ten attempts in a minute.

Master data comes next. An administrator can create a clinic and a department inside it. Department codes are unique inside a clinic. An administrator can create a receptionist or a doctor. A doctor requires a department, a registration number, and

a consultation fee stored in paisa. The registration number is unique. An administrator replaces the doctor's weekly schedule as a whole, rather than editing one cell, so the saved week is exactly the list that was submitted. An administrator or a receptionist can generate slots for a date range of at most thirty-one days. Generating the same range again does not create duplicate slots. Any signed-in user can list clinics, departments, doctors, and the open slots of a doctor on a chosen date. Open slots that have already started are not shown.

Booking is the core requirement. A patient may book only his or her own patient profile. A receptionist or an administrator may book for any patient. A doctor may not book. The slot must be open and must still be in the future, using India Standard Time, which does not observe daylight saving and is stored as a fixed offset of five hours and thirty minutes. The successful call creates an appointment in the booked state, allocates the next token number for that doctor and that date, and writes an outbox event. A second active appointment for the same patient, doctor, and date is rejected. Cancelling a future appointment returns the slot to open. Cancelling after the start time blocks the slot so it cannot be sold again. A patient may cancel only a booked appointment, and only until the configured cutoff, which defaults to two hours before the start. Reception and the administrator may cancel a booked or checked-in appointment without that cutoff.

The queue requirement is deliberately narrower than a general waiting room. Check-in is a reception action and moves a booked appointment to checked in. The doctor's call-next action selects the smallest token that is still waiting and whose appointment is checked in. A patient who booked token 1 and has not arrived is not called. If that patient checks in later, and the token is still the smallest waiting checked-in token, he or she is called next. That is an appointment-priority rule. Walk-in patients receive a new, higher token, so they join the end. A no-show, marked by reception, skips the token and releases or blocks the slot by the same time rule as cancellation. A no-show is not billed.

The clinical and billing requirements close the visit. Only the doctor who owns the appointment may complete it, and only from the in-consultation state. The completion stores a note and between one and twelve prescription lines, marks the token done, moves the appointment to completed, and creates one invoice for the doctor's consultation fee in the due state. Reception or an administrator may record a payment by cash, UPI, or card. The amount cannot exceed the remaining balance. When the paid sum reaches the invoice amount, the invoice becomes paid. Partial payment is allowed and is tested. Patients may read their own invoice. They may

not read another patient's.

Table 2.01: Functional requirements by area

Area	Requirement	Result when it fails
Identity	Patient self-registration only	Staff roles are rejected
Identity	Generic login failure	Unknown email and bad password match
Schedule	Replace the whole week	Empty block list is rejected
Schedule	Generate at most 31 days	Longer ranges return 400
Booking	Conditional slot update	Second caller receives 409
Booking	One active visit per doctor per day	Duplicate day returns 409
Queue	Call only checked-in patients	Empty waiting room returns 404
Clinical	Only the assigned doctor writes notes	Other doctors receive 403
Billing	Payment cannot exceed the balance	Overpay returns 400
Audit	Staff actions write an audit row	The admin can list recent rows

Table 2.01 is the short list a reviewer can scan. The sentences above it are the same requirements with the policy attached. The failure column is the HTTP result the service is written to produce. Validation errors, such as a missing email, return 422 with a field list. Conflicts return 409. Forbidden roles return 403. Missing records return 404.

Non-functional requirements

The non-functional requirements are the ones that decide whether the functional list survives a real morning. Correctness under overlap is the first. The booking path must not depend on a read of the slot followed later by a hopeful write. The write itself must carry the condition. The test that fires sixteen bookings at one slot is the acceptance check for this requirement. Privacy is the second. A patient must not read another patient's name from the queue, and a doctor must not call another doctor's queue. Availability of the read path is the third. Listing open slots should be able to avoid a database round trip when the list was fetched a moment ago, and

must not keep showing a slot after it has been booked and the transaction has committed.

Security requirements are modest and explicit. Passwords are hashed. Tokens expire. The production process will not start on the sample secret. Login attempts are throttled per client address. The database is not exposed as a public port in the deployment design. Operational requirements include a health route, an audit list for the administrator, and cache counters so a pilot can see whether the cache is being used. Portability is a requirement of the project as submitted: the tests use SQLite, and the schema and the Compose file show how the same model runs on MySQL. Performance numbers from SQLite are not offered as Amazon RDS numbers. They are offered as evidence that the cache path stops calling the loader.

Table 2.02: Non-functional requirements

Quality	Target in this project	How it is checked
Correctness	One winner for one slot	Sixteen overlapping bookings
Privacy	Role checks on every action	Patient cannot create a clinic
Read cost	Warm slot list skips the loader	Hit and miss counters, timings
Recoverability	Event row commits with the visit	Unpublished outbox count
Idempotent delivery	Duplicate event is stored once	Notification log unique event id
Secret hygiene	Sample JWT secret refused in production	Settings validation
Clock	Slot times use IST, events use UTC	Past slots cannot be booked
Operability	Health route and audit list	GET /health, admin audit

Table 2.02 ties each quality to a check that exists in the repository or in the settings code. A target that cannot be checked is not listed. There is no claim of a particular requests-per-second rating, because that rating was not measured on a production-shaped database.

Actors

Table 2.03: Actors and the work they are allowed to do

Actor	Creates	Must not do
Patient	Own account, own booking, own cancel inside the cutoff	See the queue or book for someone else
Receptionist	Walk-in booking, check-in, no-show, payment, slot generation	Write a clinical note
Doctor	Call next on own queue, note and prescription	Take payment or edit the weekly schedule
Administrator	Clinic, department, staff, schedule, audit	Act as the consulting doctor

Table 2.03 is the authority matrix in ordinary language. The implementation uses the same four role strings on the user row and on the token: ADMIN, RECEPTIONIST, DOCTOR, and PATIENT. A disabled user cannot sign in. There is no fifth role for a platform operator. One administrator per clinic is enough for the pilot, and that account is created by the seed or by a bootstrap step rather than by public registration.

Use cases

The use cases below are the morning written as steps. They overlap the requirements on purpose. A requirement says the rule. A use case says who starts, what is already true, and how it ends. Figure 2.01 draws the same cases as ovals inside the system boundary so the four actors are visible at once.

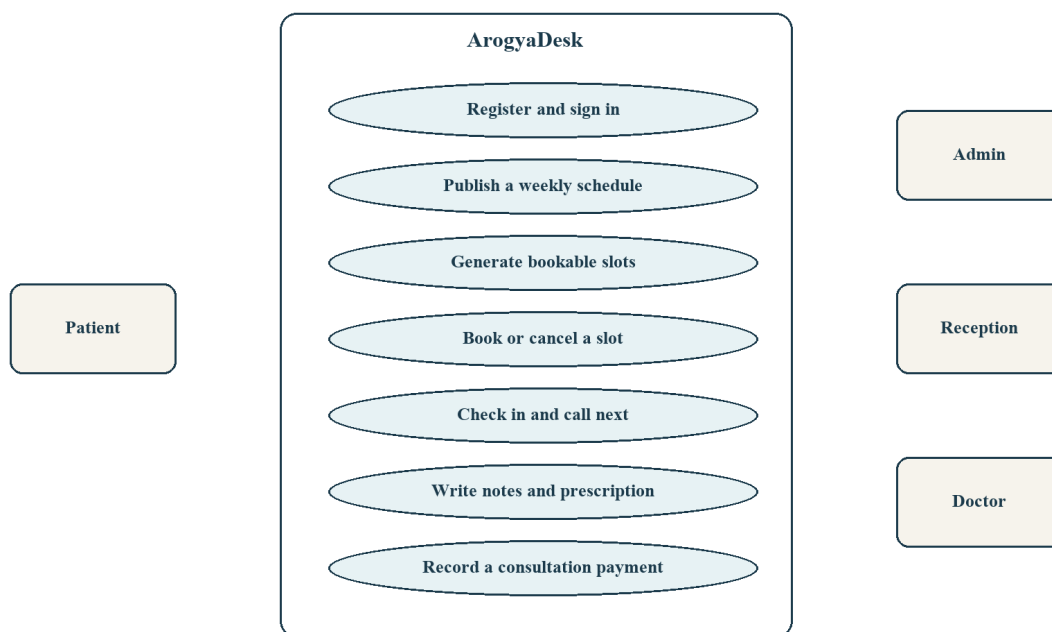


Figure 2.01: Use cases of ArogyaDesk

Use case UC-01 is registration and sign-in. The patient submits a name, email, and password. The system stores the user, stores an empty patient profile linked to that user, and returns a token. Later the patient exchanges the email and password for a new token. If the password is wrong, the message does not say whether the email exists. UC-02 is the administrator publishing a week. The administrator sends the blocks for the doctor: weekday, start, end, and slot length. The previous blocks for that doctor are removed and the new list is saved. A block whose start is not before its end is rejected.

UC-03 generates slots. Reception chooses a doctor and a from-date and to-date. For each date, the service finds the blocks whose weekday matches and walks from the start time in steps of the slot length, stopping before it would pass the end. Existing slots are counted as skipped. New slots are inserted as open with version 1. UC-04 is the booking use case and is the one developed in full in the feature chapter. The caller sends a slot id and a patient id. The service checks the role, rejects a started slot, claims the slot with a conditional update, rejects a second active visit on that day, allocates a token, and commits the appointment with an outbox row.

UC-05 is cancellation. The patient sends a reason. If the appointment is booked and the current time is still earlier than the start minus the cutoff, the appointment becomes cancelled, the token is skipped, and a future slot becomes open again. Reception can do this later in the visit, up to the checked-in state. UC-06 is check-in and call-next. Reception checks the patient in. The doctor asks for the next patient on a date. The service walks waiting tokens in number order and takes the first whose appointment is checked in. That appointment becomes in consultation and the token becomes called.

UC-07 closes the consultation. The doctor sends a note and the prescription lines. The service writes the encounter, the lines, and the invoice, and completes the appointment. UC-08 records money. Reception sends an amount and a method. The service refuses an amount above the remainder and marks the invoice paid when the remainder reaches zero. UC-09 is the no-show. Reception marks a booked or checked-in patient who will not be seen. No invoice is created. UC-10 is audit. The administrator lists recent actions. The list is not a clinical record. It is a trail of who created a clinic, who booked, who checked in, and who took a payment.

Feature set

Table 2.04: Feature set of the application

Feature	Roles	Stored result
Register and login	Public, then any user	User, patient profile, JWT
Clinic and department	Administrator	Clinic and department rows
Staff account	Administrator	User and, for a doctor, a doctor row
Weekly schedule	Administrator	Replacement weekly blocks
Slot generation	Administrator, receptionist	Open slots, idempotent
Open slot list	Any signed-in user	Cached list of future open slots
Book	Patient, receptionist, administrator	Appointment, token, outbox
Cancel and no-show	As in the matrix	Status change and slot release
Check-in and call next	Reception, then the doctor	Queue moves forward
Encounter	Assigned doctor	Note, prescription, due invoice
Payment	Receptionist, administrator	Payment rows, paid invoice
Audit and cache stats	Administrator	Recent actions, hit and miss counts

Table 2.04 is the feature catalogue. Each row is implemented by a route under /api/v1, except the health route which sits at /health so a load balancer can call it without a token. The interactive reference at /docs is generated from the same routes. A reviewer who runs the seed can sign in as the demo patient and walk the feature column without reading the service source first. The source remains the authority when the documentation and the code disagree, and the tests are the authority when a sentence in this report and the code disagree.

Assumptions and constraints

The clinic is assumed to work in India Standard Time. Slot dates and slot times are local clinic times. Event timestamps are UTC. A clinic that spans time zones is out of scope. One currency is assumed, the Indian rupee, stored as paise so that a fee is never a binary fraction. The doctor's fee is assumed constant for the pilot. A fee that changes does not rewrite old invoices, because the invoice copies the fee at the

moment the visit is completed.

The project assumes a single API process in the tests and a small number of processes behind a load balancer in the deployment design. The in-memory cache is correct for one process. More than one process requires Redis, which is why the deployment design turns Redis on. The project assumes that Kafka delivery is at least once. Consumers must tolerate a repeated event. The notification log does that with a unique event id. The project assumes that the people using the API are the four roles above. It does not assume a trustworthy client application.

Constraints from the programme are also part of the requirement story. The work had to be explainable as a backend: schema, class responsibilities, one feature followed from the request to the database, a measurement, and a cloud deployment design. The stack had to be something that could be executed and tested on the machine available for the submission. The API is therefore written in Python with FastAPI and SQLAlchemy. The deployment chapter still places MySQL, Redis, and Kafka on managed Amazon services, which is the shape the design is aiming at, whether or not the first pilot runs those processes in Docker on one host.

Class Diagrams

How the code is layered

The low-level design is a small set of classes with obvious jobs. HTTP details stay in the routers. Rules stay in the services. Rows stay in the SQLAlchemy models. The cache and the outbox are neighbours of the services, not hidden inside a controller. This is the same separation a Model-View-Controller description asks for, with the view replaced by JSON. The router is the controller. The service is the application model. The table is the persistent model. There is no server-rendered page, so there is no view object. Keeping the name 'controller' for the router would only confuse a reader who opens the repository and finds files called routers. The report uses the names in the code, and states the mapping once.

A request enters FastAPI, is validated by a Pydantic model, and is handed to a function on a router. That function builds the service with the current database session and the process cache, calls one method, and returns a response model. The session commits if the method returns, and rolls back if the method raises. Services do not commit. That choice matters for the booking tests, which open their own sessions and commit themselves, and it matters for the outbox, which must be in the caller's transaction. Raising an application error with a status code is how a service refuses work. The error handler turns that into a JSON body with a status, a short

error phrase, and a message. Unexpected exceptions are left to the server log rather than being rewritten into a false success.

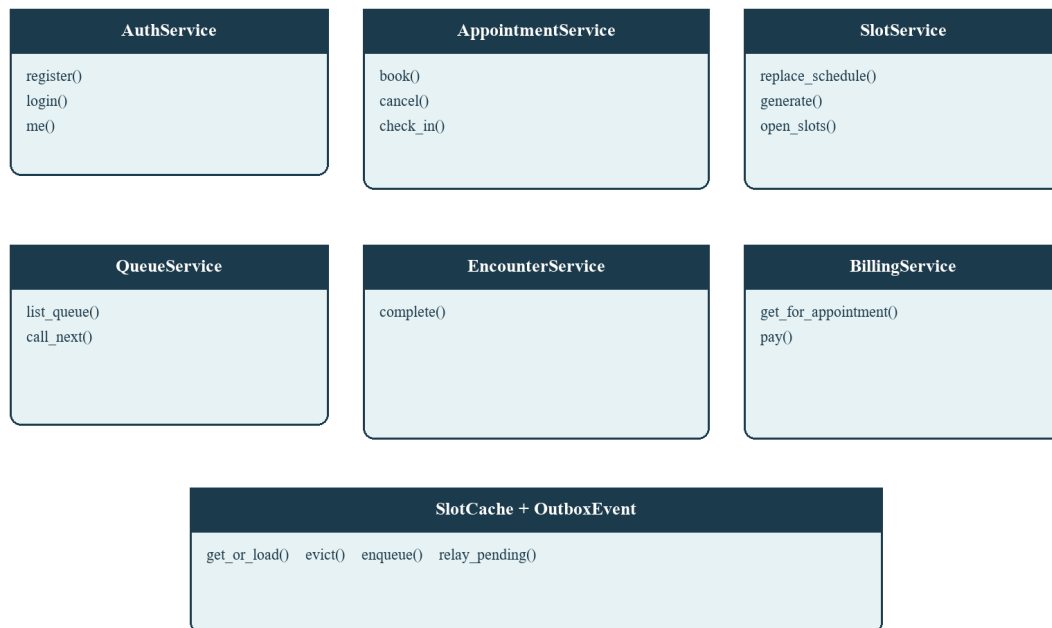


Figure 3.01: Principal classes and the methods a reviewer will look for

Figure 3.01 is not every class in the package. Models, schemas, and the token helper are omitted so the diagram stays readable. The omitted classes are simple: a model is a table, a schema is a request or a response, and the security module hashes passwords and signs tokens. The classes on the diagram are the ones that contain policy. Table 3.01 says what each of them is responsible for, and what it is careful not to do.

Table 3.01: Class responsibilities

Class	Responsible for	Leaves to someone else
AuthService	Register, login, token response	Role checks on later routes
ClinicService	Clinics and departments	Doctor fees
DoctorService	Staff users and doctor profiles	Weekly times
SlotService	Schedule replacement and slot generation	Who may book
AppointmentService	Book, cancel, check-in, no-show	Calling the next token
QueueService	List the day and call next	Writing the prescription

Class	Responsible for	Leaves to someone else
EncounterService	Note, prescription, invoice creation	Taking money
BillingService	Read invoice and record payment	Changing the fee
SlotCache	Remember or forget an open-slot list	Deciding that a slot is free
Outbox helpers	Store and publish events	The business decision to book

Table 3.01 is the rule used when a feature was added. If a change needed a new policy, it went to the service that already owned that policy. Payment limits went to BillingService. The decision to skip a patient who has not arrived went to QueueService. The decision to claim a slot went to AppointmentService. The cache was not allowed to be the authority on whether a slot was free. It is allowed only to remember a list that MySQL produced.

AppointmentService in more detail

AppointmentService is the class the rest of this report returns to. Its constructor takes a session, a cache, and the settings, because cancellation needs the cutoff hours and the cache needs to be told when a date is stale. The book method receives the signed-in user, a slot id, and a patient id. It loads the patient and the slot, checks the role, rejects a slot whose start is not in the future, and then issues the conditional update. Only after that update reports one changed row does it look for another active visit on the same day, allocate a token, and insert the appointment with a day guard. The day guard is a string of patient, doctor, and date. A unique constraint on that column is the database form of the same-day rule. When the visit ends, the guard is cleared so a later visit on another date, or a new visit after a cancellation, is not blocked by an old string.

The class also owns check-in, cancellation, and no-show, because those methods move the same appointment row and the same slot row. Splitting them into three classes would have spread the status rules. QueueService is separate because call-next is a read of many tokens and a transition that only the doctor performs. EncounterService is separate because it creates new rows rather than fighting over the slot. They still share the appointment status constants in the domain module, so a string cannot be invented in one class and misunderstood in another.

Appointment states

An appointment is a small state machine. It is created as booked. Reception may move it to checked in, or to cancelled, or to no-show. The doctor may move a checked-in appointment to in consultation by calling next, and may move in consultation to completed by saving the note. Completed, cancelled, and no-show do not move again. A patient may cancel only from booked, and only inside the cutoff. The service checks both the role and the current status. A second completion does not create a second invoice, because the status is no longer in consultation.

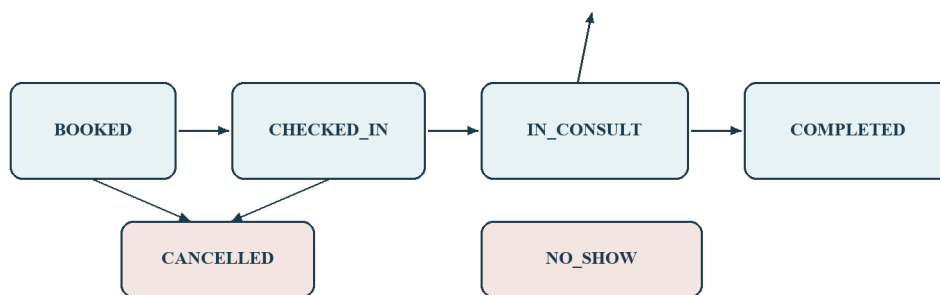


Figure 3.02: Appointment states and the transitions the services allow

Figure 3.02 shows those states. The arrows from booked and checked in toward cancelled and no-show are the release paths. They also update the slot. A future slot returns to open. A slot that has already started becomes blocked. Completed does not release the slot, because the time was used. The token follows the appointment: waiting, then called, then done, or skipped when the visit is cancelled or marked as a no-show.

Table 3.02: Allowed appointment transitions

From	To	Who
BOOKED	CHECKED_IN	Receptionist or administrator
BOOKED	CANCELLED	Patient inside the cutoff, or reception, or administrator
BOOKED	NO_SHOW	Receptionist or administrator
CHECKED_IN	IN_CONSULT	The assigned doctor, through call next
CHECKED_IN	CANCELLED	Receptionist or administrator
CHECKED_IN	NO_SHOW	Receptionist or administrator

From	To	Who
IN_CONSULT	COMPLETED	The assigned doctor

Table 3.02 is the machine as a list. There is no arrow from completed back to booked. A correction after completion is a new appointment, not an edit of history. That is stricter than some clinic software, and it is the right default for a project that is trying to keep an audit. A wrong prescription in a real deployment would need an addendum. That addendum is future work, not a silent overwrite.

Sequence of a successful booking

The sequence a reviewer should be able to trace is short. The patient sends POST /api/v1/appointments with a slot id and a patient id, and a bearer token. The router asks the dependency for the current user and for a session. AppointmentService.book runs. It updates the slot, inserts the appointment and the token, and inserts an outbox row whose payload includes the new event id. The session commits. An after-commit hook drops the cache entry for that doctor and date. The router returns 201 with the appointment, including the token number. Figure 3.03 draws that path. The cache is last on purpose. Clearing it before the commit would let a concurrent reader reload the old open list and store it again.

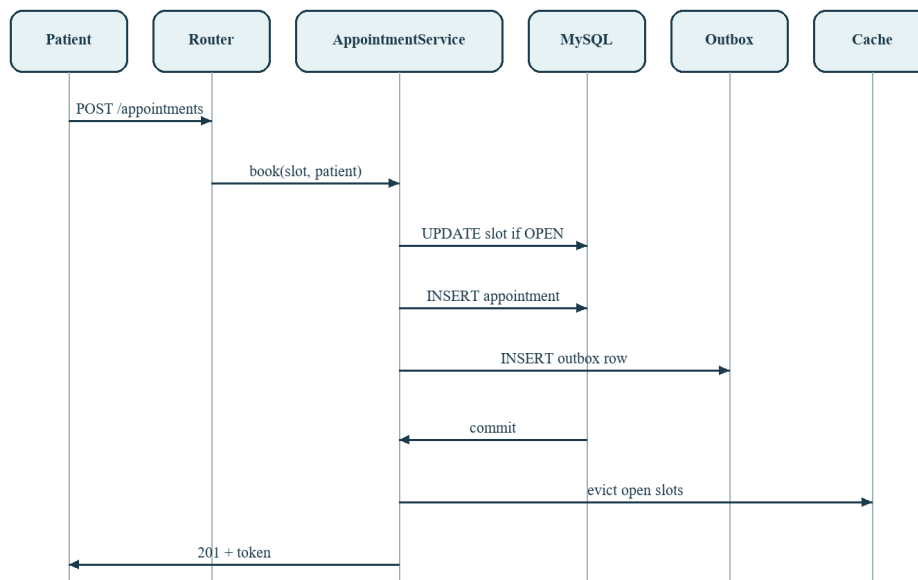


Figure 3.03: Sequence of a successful booking

If the conditional update changes no row, the service raises a conflict before any appointment is inserted. If a later step fails, the session rolls back and the slot update disappears with it. The caller sees an error. The slot remains open for a later, honest attempt. This is the behaviour the overlap test depends on. The sequence

diagram shows the happy path because the failure paths are the absence of the insert arrows, not a different design.

The cache and the outbox as classes

SlotCache is a small interface with three operations: get or load, evict, and stats. InMemorySlotCache keeps a dictionary guarded by a lock and a monotonic expiry. RedisSlotCache uses the same operations against a Redis key named with the doctor id and the date. The factory builds one or the other from the REDIS_ENABLED setting. Services depend on the operations, not on the dictionary. That is why the booking tests can run with the memory cache and the deployment design can turn Redis on without rewriting AppointmentService.

The outbox is a pair of functions rather than a large class, because the work is small. enqueue adds a row, flushes so the database assigns an id, and then stores a JSON payload that includes that id. relay_pending reads unpublished rows in id order, asks a publisher to send each payload, and sets published_at. KafkaPublisher is the production publisher. RecordingPublisher is the test publisher. The notification consumer stores a row keyed by event id and flushes, so a second delivery of the same payload does not create a second log row. The class diagram groups these under one box because their job is the same: remember that something happened, and tell the outside world without doing it inside the booking rule.

Database Schema Design

Rules that shaped the tables

The schema is relational and deliberately boring. Each fact lives in one table. Identifiers are integers. Statuses are short strings, not numbers that a reader must decode. Money is an integer number of paise. Dates of slots are dates. Start and end are times. Instants, such as when a booking was made or when a payment was taken, are timestamps. The application writes those instants in UTC. Slot walls times are interpreted as India Standard Time when the service decides whether a slot has started.

Rows are not hard-deleted in the normal flow. A cancelled appointment remains, with a reason. A blocked slot remains, so the history of that quarter-hour is still there. The unique day guard is set to null when the visit is no longer active, which releases the unique constraint without deleting the appointment. This makes the audit possible. It also means a reporting query must filter on status instead of assuming that every appointment row is a person still expected in the building.

The reference script is db/schema.sql, written for MySQL. The running tests create the same tables through SQLAlchemy on SQLite. The names, the uniqueness rules, and the indexes that the booking path relies on are the same in both. Where MySQL is stricter or richer, the script shows that form. The paragraphs below describe the tables in words, then the keys, then the cardinality, then the indexes, and finally a worked morning so the columns are not only a list.

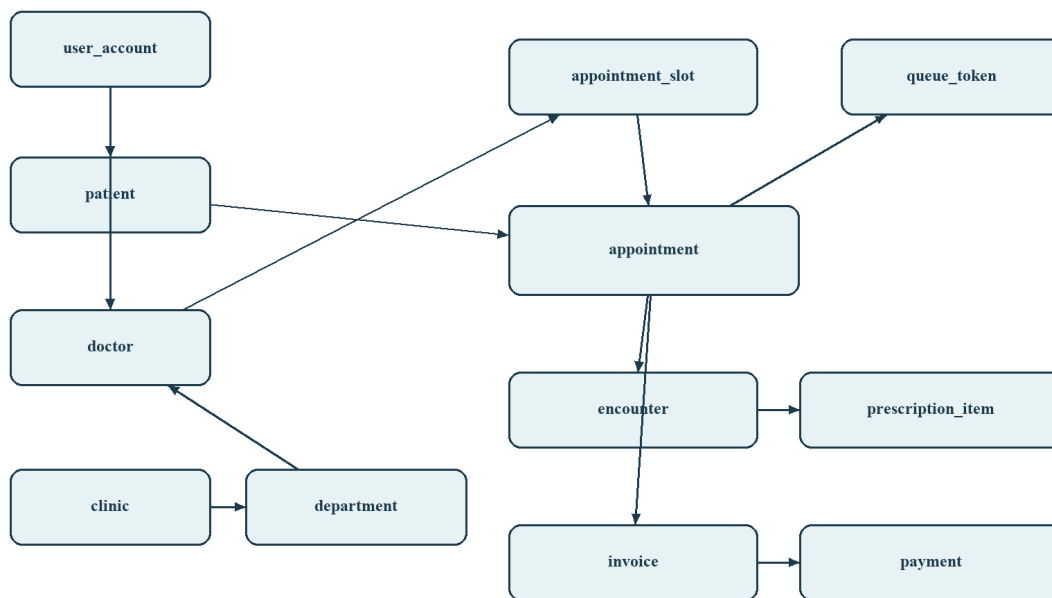


Figure 4.01: Entities and the relationships between them

Tables

user_account holds the person who can sign in: email, password hash, full name, phone, role, enabled flag, and created time. Email is unique. patient holds the clinical profile for a user whose role is patient: optional date of birth, gender, and blood group, and a unique user id. doctor holds the clinical profile for a user whose role is doctor: unique user id, department, unique registration number, consultation fee in paise, and an active flag. A receptionist has a user row and no doctor row and no patient row.

clinic holds the name, city, address, phone, and created time. department holds a clinic id, a name, and a code. The pair of clinic and code is unique, so two clinics may both have a department coded GM. weekly_schedule holds one block of a doctor's week: doctor, ISO weekday from 1 for Monday through 7 for Sunday, start time, end time, and slot length in minutes. The doctor, weekday, and start time together are unique.

appointment_slot is the scarce resource. It holds the doctor, the date, the start, the end, a status of OPEN, BOOKED, or BLOCKED, and a version integer. The doctor, date, and start are unique, so generation cannot insert the same quarter-hour twice. appointment holds the visit: slot, patient, doctor, status, token number, booked time, an optional cancel reason, and the nullable day guard. queue_token holds one row per appointment: doctor, date, token number, and a status of WAITING, CALLED, DONE, or SKIPPED. The doctor, date, and token number are unique. The appointment id on the token is unique.

encounter holds one note per appointment: the doctor, the patient, the note text, and the time. prescription_item holds the lines of that note: medicine name, dosage, duration in days, and optional instructions. invoice holds one bill per appointment: patient, amount in paise, status of DUE, PAID, or VOID, and the time it was issued. payment holds the amounts taken against that invoice: amount, method, optional reference, and time. A paid invoice is not a feeling. It is an invoice whose payments sum to at least the amount.

outbox_event holds the message that must not be lost if the process dies after the commit: aggregate type, aggregate id, event type, JSON payload, created time, and a null published time until the relay succeeds. notification_log holds what a consumer has already applied, with a unique event id. audit_entry holds actor, action, entity type, entity id, a short detail, and a time. The audit actor is not a foreign key, so a trail can still be read if user rows are later archived. Nothing in the current flow deletes users.

Table 4.01: Tables in the schema

Table	Purpose	Row appears when
user_account	Sign-in identity	Registration or staff creation
patient	Patient profile	A patient registers
doctor	Doctor profile and fee	An administrator creates a doctor
clinic	A site of care	An administrator creates a clinic
department	A unit inside a clinic	An administrator adds a department
weekly_schedule	Repeating hours	An administrator saves a week

Table	Purpose	Row appears when
appointment_slot	One bookable interval	Slot generation
appointment	One visit	A successful booking
queue_token	The patient's number	The same booking transaction
encounter	The consultation note	The doctor completes the visit
prescription_item	One medicine line	The same completion
invoice	The consultation fee	The same completion
payment	Money received	Reception records a payment
outbox_event	A durable event	Book, cancel, no-show, complete, or pay
notification_log	Applied events	The consumer sees a payload
audit_entry	Who did what	The main staff and booking actions

Table 4.01 is the map of the schema. Sixteen tables is enough for the visit and not enough for a hospital. The outbox and the notification log are included because the event path is part of the design, not an afterthought pasted onto a CRUD application.

Keys, foreign keys, and cardinality

Every table has an integer primary key. Foreign keys point at the parent that must exist first. `department.clinic_id` references `clinic`. `doctor.user_id` references `user_account` and `doctor.department_id` references `department`. `patient.user_id` references `user_account`. `weekly_schedule.doctor_id` and `appointment_slot.doctor_id` reference `doctor`. `appointment` references the slot, the patient, and the doctor. `queue_token` references the appointment and the doctor. `encounter` references the appointment. `prescription_item` references the encounter. `invoice` references the appointment and the patient. `payment` references the invoice.

The cardinalities follow from those keys and from the unique columns. A clinic has many departments. A department has many doctors. A user has at most one patient profile and at most one doctor profile. A doctor has many weekly blocks and many slots. A slot is referenced by appointments over time, but the conditional update and the application rule aim at one active booking. The schema does not put a unique

constraint on appointment.slot_id, because a cancelled appointment remains and a later patient may book the same slot. The status on the slot, not a unique appointment pointer, is what makes the slot free or taken.

A patient has many appointments across days. A doctor has many appointments. An appointment has one token, one optional encounter, and one optional invoice. An encounter has many prescription lines. An invoice has many payments. An outbox row points at an aggregate id that is an appointment, an encounter, or an invoice, depending on the event type. That pointer is not a foreign key, because the aggregate type changes. The payload carries the identifiers a consumer needs.

Table 4.02: Cardinality of the main relationships

From	To	Cardinality	Why
clinic	department	1 to many	A site has several units
department	doctor	1 to many	Several doctors can share a unit
user_account	patient	1 to 0 or 1	Only patient users have a profile
user_account	doctor	1 to 0 or 1	Only doctor users have a profile
doctor	weekly_schedule	1 to many	A week is several blocks
doctor	appointment_slot	1 to many	A day is several slots
patient	appointment	1 to many	A person returns on later days
appointment_slot	appointment	1 to many over time	History remains after cancel
appointment	queue_token	1 to 1	One number per visit
appointment	encounter	1 to 0 or 1	Only a completed visit has a note
encounter	prescription_item	1 to many	A note can list several medicines
appointment	invoice	1 to 0 or 1	A no-show is not billed

From	To	Cardinality	Why
invoice	payment	1 to many	Partial payments are allowed

Table 4.02 is the cardinality list the template asks for, written against this schema rather than against a sample that belongs to another domain. The slot-to-appointment line is the one a reader might argue with. It is one-to-many because history is kept. It behaves as one active booking because of the slot status and the conditional update. Both sentences are true, and they are true about different questions.

Indexes and the day guard

Uniqueness is an index with a rule. Email, registration number, department code inside a clinic, slot start inside a doctor-day, token number inside a doctor-day, and the day guard are all unique. The day guard deserves a paragraph of its own. While an appointment is active, the column holds a string such as 3:1:2026-10-02, meaning patient 3, doctor 1, that date. A second insert with the same string fails. When the appointment is cancelled, completed, or marked as a no-show, the column is set to null. Unique indexes allow many nulls, so old visits do not collide. The application also checks for an active visit before the insert, so the usual failure is a clear 409 rather than a raw constraint error. The constraint remains for the case where two transactions pass the check together.

The read that lists open slots filters by doctor, date, and status, then orders by start time. The index `idx_slot_doctor_date_status` is built for that filter. It does not remove the need for the cache. It makes the miss cheaper. Appointments are indexed by patient and status, and by doctor and status, for the lists a patient and a doctor open. Unpublished outbox rows are indexed by published time and id so the relay can read the oldest unpublished rows without scanning the whole history.

Table 4.03: Indexes that protect correctness or the hot read

Index	Columns	What it is for
user email	email, unique	Sign-in and duplicate registration
doctor registration	registration_number, unique	One profile per licence number
department code	clinic_id, code, unique	Codes repeat across clinics, not inside one

Index	Columns	What it is for
slot identity	doctor_id, slot_date, start_time, unique	Generation is idempotent
open slot read	doctor_id, slot_date, status	The list behind the cache miss
day guard	day_guard, unique	One active visit per patient, doctor, and date
token identity	doctor_id, queue_date, token_number, unique	Two visits cannot share a number
outbox relay	published_at, id	Oldest unpublished events
notification	event_id, unique	A repeated Kafka delivery is ignored

Table 4.03 is the index list that should be reviewed with the booking code, not as decoration. An index that nothing queries is not listed. Primary keys are omitted because every table has one and listing them sixteen times would not add a design decision.

A worked morning

The seed creates a morning that makes the columns concrete. CityCare OPD is a clinic in Hyderabad. General Medicine has the code GM. Dr. Meera Iyer is the doctor, with registration number MCI-TS-20481 and a fee of 50000 paise, which is 500 rupees. Her week, Monday to Saturday, runs from 09:00 to 12:00 in slots of fifteen minutes. Asha Rao is the demo patient. Suppose Asha books the 10:00 slot on 2 October 2026. The slot row moves from OPEN to BOOKED and its version increases. The appointment row stores her patient id, the doctor's id, the slot id, status BOOKED, token number 1 if she is first that day, and a day guard. The queue token row stores the same number with status WAITING. An outbox row stores AppointmentBooked and a payload that includes the new event id, the appointment id, and the token number.

Reception later checks her in. Only the appointment status changes, to CHECKED_IN. The token stays WAITING until the doctor calls it. When the doctor calls next, the token becomes CALLED and the appointment becomes IN_CONSULT. When the doctor saves a note for viral fever and one line for paracetamol, an encounter row and a prescription row appear, the appointment becomes COMPLETED, the token becomes DONE, the day guard becomes null, and an invoice of 50000 paise is DUE. A cash payment of 20000 leaves the invoice

DUE with 30000 remaining. A UPI payment of 30000 marks it PAID. That sequence is the visit test in the repository. The overlap test uses a different slot and shows the other outcome: fifteen of sixteen callers receive a conflict, one appointment is stored, and one outbox row is waiting.

Table 4.04: Sample rows after Asha books the 10:00 slot

Table	Important columns	Sample value
appointment_slot	status, version	BOOKED, version increased by 1
appointment	status, token_number, day_guard	BOOKED, 1, patient:doctor:2026-10-02
queue_token	token_number, status	1, WAITING
outbox_event	event_type, published_at	AppointmentBooked, null until relayed
invoice	amount_paise, status	Absent until the doctor completes the visit

Table 4.04 stops at the booking, because that is the moment the schema is doing its most important work. The invoice line is included as an absence on purpose. A booking is not yet a bill. Creating the bill at booking time would charge a patient who later cancels inside the window, and would invent a fee for a no-show, which this project refuses to do.

The conditional update in SQL

The schema supports the booking rule with an ordinary update, not with a special database product. The statement the service issues is the one below. It is generated by SQLAlchemy. The meaning is the part that matters.

```
UPDATE appointment_slot
SET status = 'BOOKED', version = version + 1
WHERE id = :slot_id AND status = 'OPEN'
```

If the database reports that one row changed, this transaction owns the slot. If it reports that none changed, someone else already booked it, or the slot was blocked, and this transaction must stop. The version column is incremented so a later reader can see that the row moved, and so a future optimistic check has a number to compare. The current code treats the status condition as the authority. The version is still updated so the row is not a static record pretending nothing happened.

On MySQL this statement takes a row lock for the duration of the transaction under the usual isolation level. On SQLite the whole database writer lock serialises the writers. The application rule is the same in both places: look at the row count, not at a value that was read earlier. That is why the project can demonstrate the rule on SQLite and still describe it as the rule that will run on RDS.

Feature Development Process

Why booking is the feature this chapter follows

Every feature in the catalogue can be described as a form that writes a row. Booking cannot. It is the feature where two honest requests are allowed to arrive together and only one of them may win. It is also the feature that the cache, the token, the audit row, and the outbox all have to survive. The rest of the visit is important, and the tests walk it, but a clinic can recover from a clumsy prescription screen more easily than it can recover from two patients holding the same chair. This chapter follows one booking from the HTTP body to the committed rows, then shows what was measured when sixteen bookings were aimed at one slot.

The implementation lives in `AppointmentService.book`, called by the appointments router. The same method serves a patient booking for herself and a receptionist booking a walk-in. The difference is the role check at the top, not a second copy of the update. That choice was deliberate. Two booking methods would eventually disagree about the day guard or about when the cache is cleared.

The request path

A booking is a JSON object with `slotId` and `patientId`. FastAPI rejects a body that is missing either field with status 422 and a list of field problems. A valid body reaches the router function. The bearer dependency reads the Authorization header, checks the signature and the expiry of the token, and loads the user. A missing or bad token is 401. The database dependency opens a session. The router constructs `AppointmentService` with that session, the process cache, and the settings, and calls `book`. Figure 5.01 is this path in the vocabulary of the template: the router plays the controller, the service plays the model of the use case, and SQLAlchemy is the way the model reaches MySQL. There is no view template. The response model is the view.

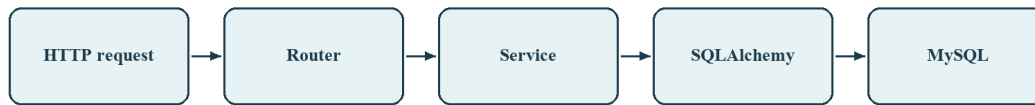


Figure 5.01: Path of a booking request through the API

On the way back, a successful call returns status 201 and the appointment: identifiers, the doctor's name, the slot date and times, the status BOOKED, the token number, and the booked time. A conflict returns 409 and a message such as 'Slot is not open' or the same-day message. A patient who sends someone else's patient id receives 403. A doctor who tries to book receives 403. A slot id that does not exist receives 404. These are not a single error with a comment inside. They are different failures and the client can show them differently.

Table 5.01: Booking request, success, and the conflict that matters

Case	Body or result	HTTP status
Request	slotId and patientId	Validated before the service
Success	Appointment with tokenNumber and status BOOKED	201
Slot taken	Message that the slot is not open	409
Same day	Message that an active visit already exists	409
Wrong role	Message that the caller cannot book	403
Started slot	Message that the slot has already started	409

Table 5.01 is the contract a front end would code against. The field names are camel case in JSON and snake case in Python. Pydantic aliases do that translation. The tests assert the camel case names, so a rename of a JSON field fails the suite instead of failing a clinic on Monday.

What the service does, in order

The method loads the patient. If the caller is a patient, the patient profile must belong to that user. If the caller is reception or an administrator, any patient id is allowed. The slot is loaded. If its start, interpreted in India Standard Time, is not strictly after the current time, the method stops. The conditional update then runs. A row count other than one becomes a conflict, and nothing has been inserted yet. The session is expired so the following read sees the booked status rather than a stale object. The same-day query looks for another active appointment for that patient and doctor on that date. Active means booked, checked in, or in consultation.

The token number is one more than the highest token already stored for that doctor and date, or 1 if there is none. The appointment is inserted with the day guard. A uniqueness failure on that guard is turned into the same-day conflict. The queue token is inserted as waiting. The outbox row is inserted for AppointmentBooked. An audit row records the actor and the token number. An after-commit listener is registered to evict the cache key for that doctor and date. The method returns the view. The session dependency then commits. Only then does the listener run. If anything raised, the dependency rolls back, the listener does not run, and the slot is unchanged.

```
claimed = session.execute(
    update(AppointmentSlot)
    .where(AppointmentSlot.id == slot_id,
           AppointmentSlot.status == "OPEN")
    .values(status="BOOKED",
           version=AppointmentSlot.version + 1)
)
if claimed.rowcount != 1:
    raise conflict("Slot is not open")
```

The excerpt above is the heart of the method, written in the shape of the Python that runs. Everything around it is either a permission check or a row that must share the fate of this update. The row count is the decision. A design that read the status into a variable, checked it in Python, and then updated by id would be wrong under overlap, even if it looked correct in a single-threaded demo.

Token, queue, and the people who have not arrived

The token is allocated inside the booking transaction so the patient can be told a number immediately. It is not allocated when the patient arrives. That choice has a consequence the queue must respect. A low token number may belong to someone who is still on the way. Call-next therefore does not take the lowest waiting token blindly. It takes the lowest waiting token whose appointment is already checked in. The people who booked early keep their priority once they arrive. The people who

have not arrived do not freeze the room. Walk-ins, booked by reception at the desk, receive the next number and therefore wait behind the numbers already issued.

This is a policy, and a clinic could want the opposite: skip a late patient permanently, or mix walk-ins by arrival time only. The code isolates that policy in `QueueService.call_next`, which walks the day's waiting tokens in order. Changing the policy means changing that walk, not changing the booking update. The report states the policy so a reviewer does not have to infer it from a loop.

Cache behaviour on this feature

The open-slot list is the read that the desk and the patient both repeat. `SlotService.open_slots` asks the cache for the doctor and the date. On a miss, the loader reads open slots, drops any whose start is not in the future, and returns plain dictionaries that can be stored as JSON. The memory cache keeps them until the time-to-live, which defaults to sixty seconds. Redis does the same with SETEX. Booking, cancellation, no-show, and slot generation evict the key after commit, so a successful write does not wait for the minute to expire.

The cache is allowed to be slightly stale for a slot that becomes past during the minute, because the loader filters by time only when it runs. Booking still refuses a started slot even if a stale list showed it. The cache is not allowed to stay stale across a commit that booked the slot. That is the invalidation rule, and it is stricter than the time-to-live. A reader who fetches in the moment between the commit and the eviction could still reload and store the old list. The window is the time of the hook. It is small, and a following read after eviction repairs it. A stricter design would version the key. That was not needed to make the tests fail closed: after the booking returns, a fresh list no longer contains the slot.

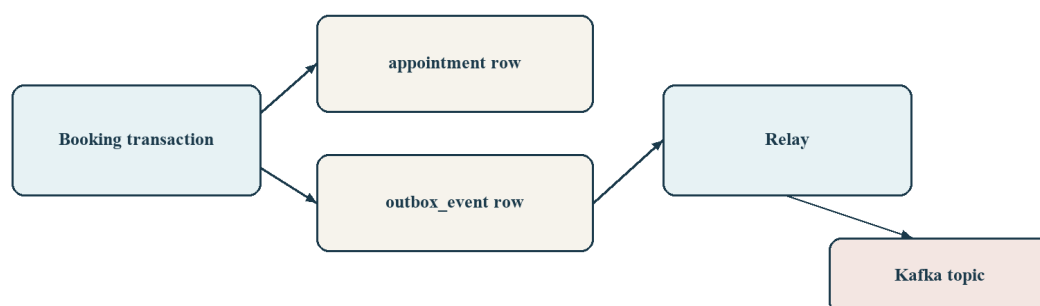


Figure 5.02: The booking transaction and the outbox row

Figure 5.02 shows why the event is a row and not a network call. The appointment and the outbox row are written together. The relay is a later reader. If the process

stops after the commit and before the relay, the row is still there with a null published time. Starting the relay again publishes it. If the relay publishes and then fails before it marks the row, it may publish twice. The consumer uses the event id so the second copy does not create a second notification. The test for that path uses an in-memory publisher, relays once, consumes twice, and expects the second consume to report that the event was already stored.

What was measured

Two measurements were taken from the automated tests on the development machine, which is a Windows workstation using SQLite. They are not a load test of Amazon RDS, and they are not presented as one. The first measurement is the overlap. Sixteen threads called book on the same future slot for the same patient. The patient and the same-day rule would also reject a second success, but the observed failures were conflicts from the slot update: the slot was no longer open. Exactly one call returned success. Exactly one appointment row was stored. The slot status was BOOKED. Exactly one outbox row was unpublished, which is what a booking writes when Kafka is off.

The second measurement is the slot list. One hundred uncached reads were timed, each of them evicting the key first so the loader had to open a session and query. One hundred cached reads were then timed with the key left in place. The average uncached read was 8.857 milliseconds. The average cached read was 0.002 milliseconds. The cached figure is a dictionary hit in the same process. It shows that the warm path does not call the loader. It does not show what a Redis hop or an RDS hop would cost. Those hops are why the deployment design still wants the cache: a warm hit should not pay the database network at all. The local numbers are the evidence that the branch exists and that the uncached branch is the one doing the query.

Table 5.02: Measurements from the test run on 28 September 2026

Measurement	Setup	Result
Overlapping bookings	16 threads, one slot, SQLite	1 success, 15 conflicts
Rows after the overlap	Count of appointments for that slot	1 appointment, status BOOKED
Outbox after the overlap	Kafka disabled	1 unpublished AppointmentBooked

Measurement	Setup	Result
Uncached slot list	100 repetitions, cache evicted each time	8.857 ms average
Cached slot list	100 repetitions, key left warm	0.002 ms average

Table 5.02 records the run. A later run on the same tests can move the milliseconds, because they include process and disk noise. The overlap result is not a timing. It is a count, and the test fails if the count is not one. Figure 5.03 draws the two averages on a logarithmic scale so the cached bar is visible beside a bar that is thousands of times taller. The caption on the figure states the scale. A linear chart would have drawn the cached bar as a line of zero height, which would have hidden the number instead of explaining it.

Average slot-list time (logarithmic scale)

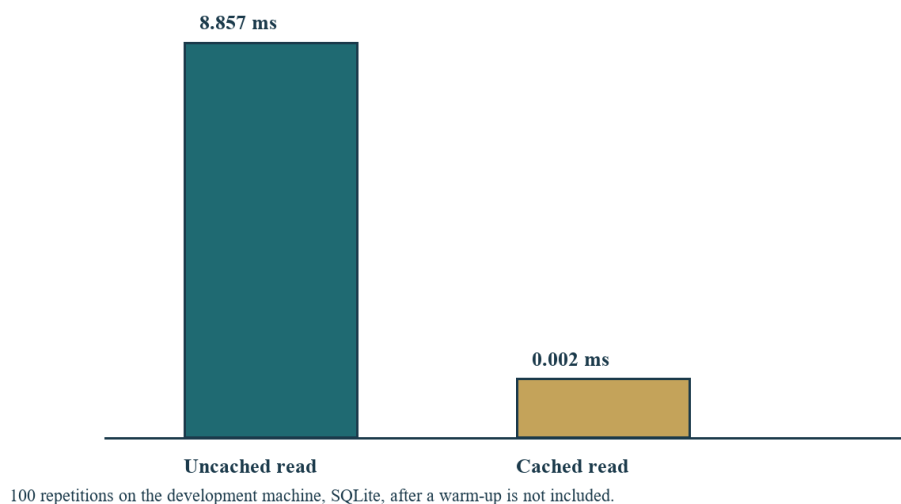


Figure 5.03: Average time to serve a slot list, logarithmic scale

Tests that hold the feature in place

The suite has ten tests. Five cover registration, duplicate email, the generic login failure, a missing token, and a patient who tries to create a clinic. One covers schedule, generation, the cache counters, a booking, and the disappearance of that slot from the next list. Two cover the visit: the path from booking through a partial payment to a paid invoice, and a cancellation that puts a future slot back on the open list. Two cover the overlap and the outbox relay. Together they are the regression net for the rules in this chapter. They are not a substitute for a clinic pilot. They are the reason a later edit to the update statement cannot quietly start creating two appointments.

Table 5.03: Automated tests and the rule each one protects

Test	Rule
Register and read profile	A patient receives a token and a patient id
Duplicate email	The second registration is a conflict
Login failure	The message does not reveal whether the email exists
Missing token	Clinic lists are not public
Patient creates a clinic	The role check returns forbidden
Slot list and booking	Generation, cache hits, and removal after booking
Full visit	Check-in, call next, note, partial pay, paid invoice
Cancel	A future slot is open again and can be seen in the list
Sixteen bookings	One appointment is stored
Outbox relay	One publish, and a duplicate consume is ignored

Table 5.03 names the tests in the language of the rules rather than in the language of the file names. The file names are `test_auth`, `test_slots`, `test_visit`, and `test_concurrency`. A reviewer runs `pytest` from the repository root. The first run on a new machine installs the requirements into a virtual environment and does not need Docker.

What was optimised, and what was refused

The optimisation in this feature is the removal of repeated slot-list queries, plus the index that makes a miss a narrow read, plus the decision not to call Kafka inside the request. Each of those has a failure mode that was treated as part of the design. The cache is invalidated on commit. The index is not trusted to prevent a double write; the conditional update is. The outbox accepts a delay of a second or two before a notification, in exchange for not losing the event when the broker is down. A design that optimised the request by skipping the row count, or by booking in the cache and writing the database later, was refused. That design is faster until the process dies, and then it is a double booking or a lost one.

Query shape was kept simple. The open-slot query filters by the indexed columns. The same-day check is a small query by patient, doctor, and active status. Token allocation reads a maximum. None of these is a reporting scan. There is no N+1 list of clinics hiding in the booking path. The place a future optimisation would start, after a pilot, is the token maximum under many different slots booked at once for one doctor. The unique token constraint would then reject a clash, and the service would need a short retry. The single-slot overlap test does not exercise that clash, because only one transaction inserts a token. The report states that limit instead of claiming the token allocator is proven under every race.

Deployment Flow

What this chapter is

This chapter describes how ArogyaDesk is meant to run on Amazon Web Services in the Mumbai region, ap-south-1, close to the clinics it is written for. The repository contains the Dockerfile, the Compose file for a local MySQL, Redis, and Kafka, and the environment variables the process reads. A live AWS account was not provisioned for this submission. There is no production URL and no claim of a measured cloud latency. The value of the chapter is the placement of each component, the security groups, and the order of a release. A reviewer can compare that design with the code: the API is a stateless process, the system of record is MySQL, the cache is Redis, and the events leave through the outbox toward Kafka.

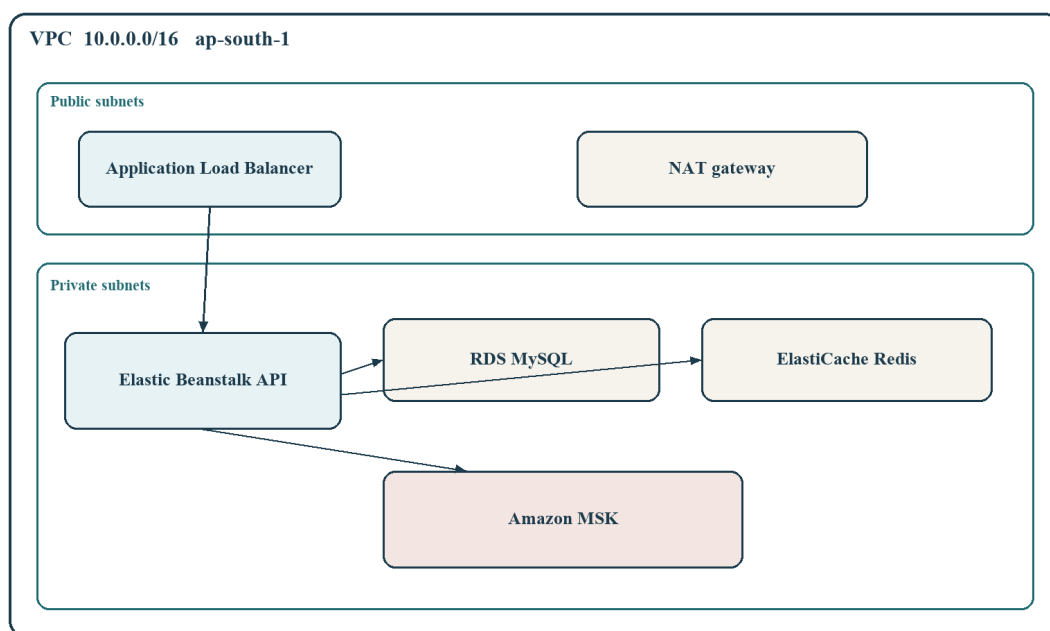


Figure 6.01: Deployment design on AWS in a single region

Figure 6.01 places the load balancer and the NAT gateway in public subnets, and the API, the database, the cache, and the broker in private subnets. Users reach only the load balancer. The API reaches the internet, for example to pull an image, through the NAT gateway. The database has no public address. That drawing is the whole security argument in one picture. The paragraphs below name the services the template asks for.

VPC, subnets, and security groups

The virtual private cloud is a /16 network, drawn as 10.0.0.0/16. Two public subnets and two private subnets sit in two availability zones so that a zone failure does not remove the only load balancer or the only API task. The database is Multi-AZ in the design, which is an RDS setting rather than a second application. The cache and the broker are sized for a pilot, not for a national chain. A single region is an explicit limit. A clinic in another region would add latency or a second deployment. Cross-region replication is not part of this design.

Security groups are the firewall. The load balancer group accepts 443 from the internet. The API group accepts traffic only from the load balancer group, on port 8080. The database group accepts 3306 only from the API group. The cache group accepts 6379 only from the API group. The broker group accepts the Kafka port only from the API group. No group uses a wide-open rule for convenience and then promises to tighten it later. The administrator's laptop is not a permanent exception. Schema changes and the rare manual query go through a bastion or a session manager in a private subnet, not through a public MySQL port.

Table 6.01: Security groups in the deployment design

Group	Inbound	From
Load balancer	443	The internet
API tasks	8080	The load balancer group only
RDS MySQL	3306	The API group only
ElastiCache	6379	The API group only
MSK	Kafka broker port	The API group only

Table 6.01 is the rule set. Outbound from the API is allowed toward those groups and toward the NAT gateway for image pulls and for any future SMS provider. The database does not need a route to the internet. Turning REDIS_ENABLED and KAFKA_ENABLED on in this network is safe only because those ports are not

public. The same flags on a laptop pointed at localhost are the development setup in the README.

EC2, Elastic Beanstalk, and the image

The API process is uvicorn serving the FastAPI application on port 8080. The Dockerfile starts from a slim Python 3.11 image, installs the requirements, and copies the app package. That image is what Elastic Beanstalk would run in a Docker environment, or what an EC2 Auto Scaling group would run under ECS. Elastic Beanstalk is the managed choice for this project because a clinic pilot does not need a platform team. Beanstalk replaces instances, rolls an environment, and attaches the load balancer. The instances themselves are still EC2. They sit in the private subnets and do not have public IP addresses. Health checks call GET /health, which does not require a token and returns a small JSON body.

The process is stateless apart from the in-memory cache, and the production settings turn that cache off in favour of Redis. Any instance can serve any request. A rolling deploy therefore does not have to drain a sticky session. The JWT secret, the database URL, and the Redis URL are environment variables, not files baked into the image. In this design they come from a secrets manager or from Beanstalk environment properties that are not committed. The sample secret in the repository is refused when ENVIRONMENT is production. That check is the last line of defence against a deploy that copied the example file.

RDS, ElastiCache, and MSK

Amazon RDS for MySQL 8 is the system of record. The parameter set uses utf8mb4. Automated backups are on. The master password is not the Compose password from the local file. The application user can read and write the ArogyaDesk schema and cannot administer the instance. Connections use the private address. The SQLAlchemy URL is the same shape as the local MySQL URL in the README, with a different host and secret. Flyway is not used. The application creates tables on startup from the models, and db/schema.sql is the reviewed statement of those tables for a database administrator who wants to create them ahead of the process. A pilot can choose either path. It should not choose both on a database that already has data and then be surprised by a difference.

ElastiCache for Redis holds the open-slot keys. The key format is open-slots, the doctor id, and the date. The time-to-live is the cache setting, and eviction on commit still deletes the key. A cache node failure empties the optimisation and does not

empty the clinic. The next read misses and loads from RDS. Amazon MSK is the Kafka service. The API produces to the topic named by `KAFKA_TOPIC`, defaulting to `arogyadesk.appointments`. A consumer group, `arogyadesk-notifications`, writes the notification log. For a pilot, a single broker is enough and is a single point of failure for notifications, not for bookings. Bookings are safe when the broker is down because the outbox row remains. The relay logs the failure and tries again.

The order of a release

A release starts with the tests on a clean checkout. The image is built and pushed to a private registry. The database, cache, and broker already exist and are not recreated by the application deploy. Environment variables are checked, including a secret that does not contain the sample text. Beanstalk, or the Auto Scaling group, starts new tasks. The load balancer sends them traffic only after `/health` succeeds. Old tasks are drained. A smoke check signs in as a non-production administrator and reads the clinic list. It does not book a real patient as a test. Seed data is off in production. The first administrator is created by a one-off bootstrap, not by the demo passwords in the README.

Rollback is the previous image. Because the schema in this project is additive across the submitted version, a rollback of the process does not require a rollback of the tables. A later change that renames a column would need a migration plan this submission does not yet have. That is a known limit of using create-all as the schema tool. The SQL file is the place a migration would start.

An illustration of cost, not a quote

Cloud cost depends on the region, the term, and the hour. The figures below are a planning illustration for a small pilot in Mumbai, using modest instance sizes, so a reader can see which component dominates. They are not an AWS invoice, not a price-list extract, and not a promise. A current quote should be taken from the AWS pricing pages before anyone approves a budget. The shape of the illustration is still useful: the database and the broker cost more than the smallest API instances, and the NAT gateway is a quiet line that people forget.

Table 6.02: Illustrative monthly cost for a small pilot, not a quotation

Component	Planning size	Why it is in the design
Elastic Beanstalk on small EC2 instances	Two small instances	Stateless API behind the load balancer

Component	Planning size	Why it is in the design
Application Load Balancer	One	The only public entry
RDS MySQL	A small Multi-AZ class	The system of record
ElastiCache	A small node	Shared slot cache for every API task
MSK	A small broker setup	Notifications, not the booking itself
NAT gateway	One per zone used	Private tasks still need outbound patches

Table 6.02 refuses invented rupee or dollar amounts. A table of precise prices that were not copied from a current price list would look more official and would be less honest. The design choice the cost affects is real: a single-clinic pilot can run MySQL, Redis, and Kafka with Docker on one host and skip MSK until there is a second process that must receive events. The code already allows that, because Kafka can be off and the outbox still fills. The AWS design is the step up, when the clinic wants a managed disk, a private network, and more than one API task.

Technologies Used

Why these tools, in this project

The tools were chosen because each one carries a job the clinic workflow actually has, and because each one can be pointed at with a setting. Python and FastAPI implement the API and the tests. SQLAlchemy talks to SQLite in tests and to MySQL in the deployment design. Redis remembers a list. Kafka carries an event after the database has committed. JSON Web Tokens identify the caller. Docker packs the process and, in Compose, packs the local stores. Amazon Web Services is the deployment design in the previous chapter. Pydantic checks the JSON. Pytest runs the rules. None of these was added so the technology chapter would have a fashionable name.

Table 7.01: Technologies and the job each one has in ArogyaDesk

Technology	Job in this project	Where a reader can see it
Python 3.11	Language of the API and the tests	app/ and tests/
FastAPI	Routes, validation, OpenAPI	app/routers, /docs

Technology	Job in this project	Where a reader can see it
SQLAlchemy	Models and the conditional update	app/models.py, AppointmentService
MySQL	System of record in the deployment design	db/schema.sql, Compose
SQLite	The database the tests actually run	Temporary file per test
Redis	Shared open-slot cache when enabled	app/cache.py
Kafka	Event transport when enabled	app/outbox.py, app/factory.py
JWT and PBKDF2	Sign-in	app/security.py
Docker	Image and local stores	Dockerfile, docker-compose.yml
AWS design	Private network and managed stores	Chapter 6

Table 7.01 is the index. The following sections say what each main technology is doing here and where the same kind of tool shows up in other systems. The comparisons are illustrations of the tool, not claims that those companies run this code.

Python and FastAPI

Python was the practical choice for a project that had to be executed and tested on the machine used for this submission, and it is a common language for service APIs. FastAPI turns a function and a Pydantic model into a route, rejects a bad body before the service runs, and publishes an OpenAPI document at /docs. That document is how a front-end developer, or a reviewer, can see the booking body without reading the router first. The dependency system is what injects the session and the current user. It is also what made the role checks a one-line declaration on the staff routes.

In industry, teams that publish internal HTTP services in Python use FastAPI or a close neighbour when they want typed request bodies and an automatic contract. The pattern is the same as a Spring controller with a validated DTO: a bad payload never reaches the rule. The lesson for this project is not the brand of the framework. It is that validation, authentication, and the rule live in different layers, which is what Chapter 3 described. A clinic that later rewrote the same rules in another

language could keep the schema and the tests' expectations and replace the routers.

SQLAlchemy and MySQL

SQLAlchemy is the library that turns a Python assignment into an INSERT and, more importantly, turns the booking rule into an UPDATE with a WHERE clause. The models declare uniqueness and the open-slot index so the test database and a MySQL database created from the models agree. MySQL is the deployment database because it is the relational engine the schema was written against, it is widely operated, and RDS offers it as a managed service with backups and a private network. SQLite is the test database because it is embedded and because the booking rule can be expressed as a conditional update there as well. The report keeps those two roles separate so a fast SQLite test is not described as a MySQL benchmark.

Relational databases of this kind sit under a large share of ordinary business software. WordPress stores its content in MySQL. Banks and retailers use other relational engines for the same reason this project does: a transaction can make several rows appear together or not at all. The appointment, the token, and the outbox row are that pattern at clinic scale. The day guard is a uniqueness constraint used as a business rule. Both techniques are older than this project, and both are the reason the overlap test can be strict.

Redis

Redis is an in-memory key store. ArogyaDesk uses it for one kind of key: the open slots of one doctor on one date, with an expiry. The application deletes the key when a booking or a cancellation commits. Redis is not asked to be the authority on the slot. If it is empty, wrong, or unreachable, the correct response of a careful operator is to turn the flag off or to fix the node, and let reads fall through to MySQL. The memory implementation used in tests has the same three methods, so the service does not contain a branch full of Redis commands.

Large consumer systems have published their use of Redis as a cache beside a durable database, for counters, sessions, and feeds. The lesson that transfers is the pairing. A cache without an invalidation story is a second database that disagrees with the first. The measurement in Chapter 5 shows the local effect of a hit. The deployment reason is the shared hit across several API tasks, which a process-local dictionary cannot provide. That is why Redis appears in the AWS design even though the tests pass without it.

Kafka and the outbox

Kafka is a log. Producers append records. Consumers read them at their own pace and remember an offset. ArogyaDesk produces appointment, encounter, and payment events, and a consumer records that it has seen an event id. The producer inside the request would be the wrong place to call the broker, because the broker can be down when the database is up. The outbox table, described by Martin Kleppmann and used widely since, is the pattern this project copies in miniature: write the message in the same transaction as the state, and publish afterwards. The relay in the factory runs only when Kafka is enabled. The tests call the same relay function with a publisher that stores messages in a list.

Kafka was built at LinkedIn and has since been described in public engineering writing from companies that move orders, trips, and playback events between services. A clinic does not have that volume. It has the same shape of problem at a smaller size: the booking must stand even if the notification is late. SMS reminders, a display screen in the waiting room, or a nightly report can all read the topic later without being called from AppointmentService. That is the practical reason to keep the relay, not a desire to operate a large cluster on day one. The cost chapter already says a pilot can leave the broker off.

Passwords and JSON Web Tokens

Passwords are hashed with PBKDF2-HMAC-SHA256 and a random salt, using the standard library. The iteration count is a setting, high by default and lower in the tests so the suite stays quick. The hash string stores the algorithm, the count, the salt, and the digest, so a later increase in iterations does not invalidate old passwords. The login comparison uses a constant-time compare. A JSON Web Token, signed with HMAC-SHA256, carries the user id, the role, and the email, and an expiry. The server still loads the user on each call, so a disabled account stops working before the token expires. The signing secret is the setting that production refuses to leave as the sample.

Tokens of this kind are common in first-party APIs and in OAuth-style designs. The OWASP authentication guidance is the checklist this project only partly meets: hashed passwords, a generic login error, and a throttle are in, and multi-factor authentication, breach detection, and a full session-revocation list are not. The gap is stated so the token is not described as a complete identity system. For a clinic pilot behind a reception desk, the larger practical risk is a shared receptionist password, which is an operating rule as much as a code change.

Docker and the cloud services

Docker gives the API a known base image and gives a developer a local MySQL, Redis, and Kafka without installing them by hand. Compose is not the production orchestrator. It is the local picture of the same three stores. The AWS services in Chapter 6 are the managed form of that picture: EC2 under Beanstalk for the process, a VPC and security groups for the network, RDS for MySQL, ElastiCache for Redis, and MSK for Kafka. Using managed stores is the point. A clinic should not become a database operator in order to stop double booking.

Other industries use the same split. A retailer may run an API on a container service and a relational database on a managed engine. A logistics firm may publish shipment events to a log and cache the tracking page. The names change. The split in this project is the one already argued: durable rows in MySQL, a disposable list in Redis, a delayed event in Kafka, and a stateless API that can be replaced by a deploy. Chapter 6 is the concrete network. This section is the reason those products were the ones named.

Conclusion

What this project taught

The useful lesson was not the list of libraries. It was the order of decisions. The slot had to be a conditional write before it was a screen. The token had to be allocated in the same transaction as the slot or the patient would hear a number that the rollback had erased. The cache had to be cleared after commit or it would remember a free slot that was no longer free. The event had to be a row before it was a message. Each of those is a small sentence and an easy bug. The tests exist so the sentences stay true when the code is edited.

The second lesson was about roles. A clinic is not a set of users with the same menu. The doctor, the receptionist, the patient, and the administrator are different authorities. Putting that in the service, and proving it with a test where the patient cannot create a clinic, is more important than a front end that hides a button. The third lesson was about honesty in measurement. Sixteen overlapping calls and a count of one are a result. A pair of millisecond averages on SQLite is a result about a cache branch. Neither number is a capacity plan for a hospital group, and the report does not dress them as one.

Where the same design is useful

The immediate use is a single-site or few-site outpatient clinic that wants one answer to the question of whether a time is free, a number the patient can see, and a fee that matches the doctor's configured charge. The same shape fits a diagnostic collection centre that books phlebotomy chairs, or a vaccination desk that books a nurse's table, if the clinical note is simplified or removed. The schema already allows more than one clinic and more than one department. A second site is data, not a fork of the program, as long as the sites can share one database and one notion of India Standard Time.

Outside clinics, the booking rule is the same rule a tutor uses for a counselling slot or a workshop uses for a bay: a scarce interval, a conditional claim, a person who may hold only one active claim, and a notification that can be late. The prescription and the invoice would not transfer unchanged. The update statement and the outbox would. That is the part worth keeping if the domain changes.

Limits and what should be done next

The limits are concrete. There is no payment gateway, so UPI in this system is a recorded reference, not a captured charge. There is no SMS reminder. There is no front end. Medicines are not priced and not stocked. A no-show is free, which some clinics will dislike. The cancellation cutoff is a single setting, not a per-doctor policy. Token allocation has not been proven under a race of many different slots at the same doctor, only under many callers on one slot. The schema tool is create-all plus a SQL file, which is weak once columns start changing. Multi-factor authentication is absent. The in-memory cache must not be used when more than one API process runs. The AWS chapter is a design, not a deployment log. The timing numbers are from SQLite on one workstation.

The next work, in order, would be a small front end for reception, a migration tool before the second schema change, a retry around token insertion, Redis in any multi-process pilot, and only then a gateway and an SMS consumer on the topic. A clinical addendum for a wrong prescription should be designed before anyone is tempted to edit a completed note. None of that work changes the booking rule this report has argued. It sits on top of it.

ArogyaDesk, as submitted, is a tested API for the outpatient visit: a slot that one person can claim, a queue that calls people who have arrived, a note, and a bill that cannot be overpaid. The repository contains the code, the schema, the tests, and the local infrastructure file. This report contains the requirements, the classes, the tables, the booking path, the measurements, and the network the process is designed

to enter. That is the whole of the claim.

References

FastAPI documentation. <https://fastapi.tiangolo.com>. Accessed 28 September 2026.

SQLAlchemy 2 documentation. <https://docs.sqlalchemy.org>. Accessed 28 September 2026.

Pydantic documentation. <https://docs.pydantic.dev>. Accessed 28 September 2026.

MySQL 8 reference manual. <https://dev.mysql.com/doc/>. Accessed 28 September 2026.

Redis documentation. <https://redis.io/docs/latest/>. Accessed 28 September 2026.

Apache Kafka documentation. <https://kafka.apache.org/documentation/>. Accessed 28 September 2026.

Jones, M., Bradley, J., and Sakimura, N. JSON Web Token (JWT), RFC 7519. Internet Engineering Task Force, 2015. <https://datatracker.ietf.org/doc/html/rfc7519>. Accessed 28 September 2026.

Python documentation, hashlib. <https://docs.python.org/3/library/hashlib.html>. Accessed 28 September 2026.

OWASP Cheat Sheet Series, Authentication Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html. Accessed 28 September 2026.

Kleppmann, Martin. Designing Data-Intensive Applications. O'Reilly Media, 2017. Consulted for the transactional outbox idea and for the distinction between a system of record and a cache.

Amazon Elastic Beanstalk Developer Guide. <https://docs.aws.amazon.com/elasticbeanstalk/>. Accessed 28 September 2026.

Amazon Relational Database Service User Guide. <https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/>. Accessed 28 September 2026.

Amazon ElastiCache User Guide. <https://docs.aws.amazon.com/AmazonElastiCache/latest/dg/>. Accessed 28 September 2026.

Amazon Managed Streaming for Apache Kafka Developer Guide. <https://docs.aws.amazon.com/msk/>. Accessed 28 September 2026.

Amazon Virtual Private Cloud User Guide. <https://docs.aws.amazon.com/vpc/>. Accessed 28 September 2026.

OpenAPI Specification. <https://spec.openapis.org/oas/latest.html>. Accessed 28 September 2026.

Cursor, AI-assisted editor. Used to help structure this report and to review the implementation. The measurements in Table 5.02 were produced by running the project tests locally. Accessed 28 September 2026.